LEGO SPIKE Prime Python

Connecting Software and Hardware with LEGO SPIKE Prime

Lecture #1: Importing Libraries - LEGO SPIKE Prime Python

SOTC 2026 - John Muir

Partner Work & Expectations

Working in Pairs Today:

- 2 students = 1 SPIKE Prime set
- Both partners contribute to coding
- Both partners submit their OWN code

Good Teammate Qualities:

- Take turns typing
- Explain your thinking
- Listen to your partner's ideas
- Share the work equally
- Help each other troubleshoot

Materials Management:

- Keep parts organized
- Return everything to correct tray slots
- Both partners help clean up

The Recipe Analogy

Cooking a Recipe:

- Buy ingredients from store
- Gather tools: stove, mixer, spatula
- Have everything **BEFORE** you start

- Buy ingredients from store
- Gather tools: stove, mixer, spatula
- Have everything **BEFORE** you start

Python Programming:

- Import libraries = gathering your tools
- Libraries link software to hardware
- Must import **BEFORE** writing code

- Import libraries = gathering your tools
- Libraries link software to hardware

- Must **import BEFORE** writing code

Python Import Syntax

Critical Rules:

- Capitalization matters: `Light_Matrix` ≠ `light_matrix`
- Punctuation matters: every comma, colon, underscore
- Imports go at the TOP of your program
- Each library imported only ONCE

Activity: Find your SPIKE hardware pieces and match them to library names

Visual: Color-coded syntax diagram

Visual: Color-coded syntax diagram

Every SPIKE Program Has This Structure

Think of it Like This:

1. IMPORTS (Lines 1-2) = Getting Your Tools Ready

- These tell Python what hardware you're using
- **YOU CHANGE THESE** based on what you need

2. MAIN FUNCTION (Lines 4-6) = Your Work Area

- `async def main()`: opens your workspace
- **YOU WRITE YOUR CODE HERE** (inside the indented area)
- This is where the action happens!

3. START BUTTON (Line 8) = Press Go!

- `runloop.run(main())` starts your program
- **YOU DON'T CHANGE THIS** - it's always the same

python

```python
from hub import light_matrix    # 1. IMPORTS (your tools)
import runloop

async def main():               # 2. MAIN FUNCTION (your workspace)
    # write your code here
    await light_matrix.write("Hi!")

runloop.run(main())            # 3. START BUTTON (runs your code)
```

Partner Activity:

- Which hardware is imported in this example? (light_matrix)
- Where would you write new code? (line 6, replacing the comment or adding after)

Hub Connection & Getting Started

Go to spike.legoeducation.com

Connect Your Hub:

- Green ring = USB connected ✓ (best)
- Blue ring = Bluetooth connected ✓
- Yellow icon = Disconnected ❌

Location: Upper left corner of SPIKE App →

First Program Steps:

1. Open SPIKE App → New Python Project
2. Connect hub (check for green/blue ring)
3. Click play button (yellow circle, bottom)
4. Watch 'Hi!' scroll on light matrix

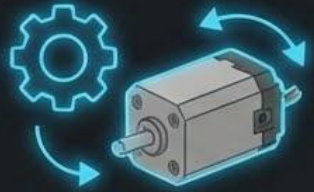
Today's Assignment - Four Programs

Create FOUR different programs with your partner.
EACH PERSON submits their own code to Google Classroom.

1. Program A: LED Matrix Display

- Change the message on light_matrix
- Display your name or custom message

2. Program B: Motor Control

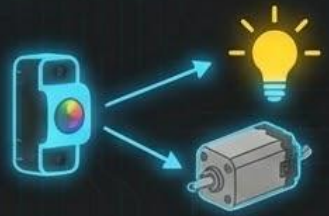


- Make a motor turn/spin
- Control speed or duration

3. Program C: Color Sensor Values

- Read and display color sensor data
- Show what the sensor detects

4. Program D: Color Sensor Action

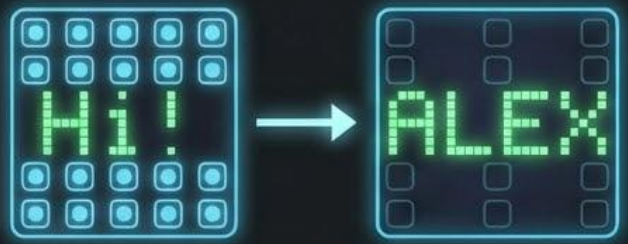


- Use color sensor to trigger something
- Example: See red → motor spins OR display changes

Program A - Your First Assignment

Program A: LED Matrix Display

- **Goal:** Change the message on the light_matrix
- **Task:** Display YOUR NAME or a custom message



With Your Partner:

1. Look at the starter code
2. Find the line: `await light_matrix.write("Hi!")`
3. Change "Hi!" to your name or a short message
4. Test it on your hub (click play ▶)
5. Does it work? Great!

File Naming: `LastName_FirstName_A_Matrix.py`

Important: We will ALL practice submitting Program A to Google Classroom before moving on!

Submit Program A to Google Classroom

Let's Practice Submitting Together:

Step 1: Open Google Classroom

- Go to [Google Classroom w4m3nvl6] + Join
- Find "Importing Libraries – LEGO SPIKE Prime" assignment
- Click on the assignment

Step 2: Upload Program A

- Click "Add or create" → "File"
- Find and select your `LastName_FirstName_A_Matrix.py` file
- Click "Upload"
- See your file appear? ✓

Step 3: Submit then unsubmit

- You need to add 3 more files later
- Just close Google Classroom for now

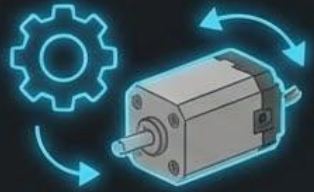
Today's Assignment - Four Programs

Create FOUR different programs with your partner.
EACH PERSON submits their own code to Google Classroom.

1. Program A: LED Matrix Display

- Change the message on light_matrix
- Display your name or custom message

2. Program B: Motor Control

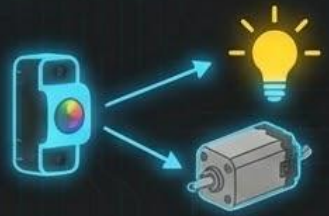


- Make a motor turn/spin
- Control speed or duration

3. Program C: Color Sensor Values

- Read and display color sensor data
- Show what the sensor detects

4. Program D: Color Sensor Action



- Use color sensor to trigger something
- Example: See red → motor spins OR display changes

Slide: Quick Reference (Leave Displayed)

Google Classroom: w4m3nvl6

Before You Leave:

✅ **ALL 4 programs uploaded to Google Classroom?** ✅ **Clicked "Turn in" button?** ✅ **Files named correctly?** ✅ **Materials organized?** All parts in correct tray locations ✅ **Hub charging?** Plug in for next class

Program Structure Template:

```python
# SECTION 1: IMPORTS (Get your tools)
from hub import _______     # Change based on hardware
import runloop              # Always needed

# SECTION 2: MAIN FUNCTION (Your workspace)
async def main():
    # YOUR CODE GOES HERE (indented)
    # Write what you want to happen

# SECTION 3: START BUTTON (Always the same)
runloop.run(main())
```

Program B - Motor Example:

```python
from hub import port
import motor
import runloop

async def main():
    # Spin motor on port A
    motor.run(port.A, 1000)  # 1000 = speed

runloop.run(main())
```

Program C - Color Sensor Example:

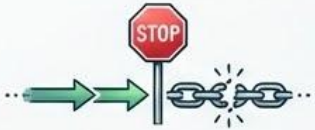
```python
from hub import port, light_matrix
import color_sensor
import runloop

async def main():
    # Read color from sensor on port B
    color = color_sensor.color(port.B)
    await light_matrix.write(str(color))

runloop.run(main())
```

Introduction to `while True:` - The Continuous Loop

Conceptual Flow & Analogies

Without `while True`:

- ❌ Your program reads the sensor ONE time
- ❌ Then the program ends
- ❌ You'd have to restart it every time



With `while True`:

- ✅ Your program keeps checking the sensor
- ✅ It responds to changes immediately
- ✅ Like a guard who never stops watching

Real-World Example

Think about a traffic light:

- → It doesn't check cars just once and stop
- → It continuously checks: "Are cars waiting? Change to green!"
- → It loops forever: check → respond → check → respond

Your color sensor program:

- Continuously checks: "What color do I see?"
- Responds based on the color
- Then checks again immediately

Python Code Implementation

Example with Color Sensor

```python
from hub import light_matrix, port
import color_sensor
import runloop

async def main():
    while True:  # Keep checking forever
        # Read the color sensor
        detected_color = color_sensor.color(port.B)

        # Display what color we see
        await light_matrix.write(str(detected_color))

        # Small delay
        await runloop.sleep_ms(500)

        # Loop automatically goes back to top!

runloop.run(main())
...
```

Mastering Decisions: The `if', `else', and `elif' Statements

1. The Basics: `if` and `else`

What is an `if` statement?

- Think: "IF this is true, THEN do this action"
- Makes decisions based on conditions

```
if condition:
    # Code here runs ONLY if true
    # Must be indented!
```

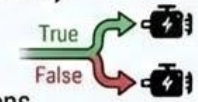
Colon : after condition

Indentation is Critical!

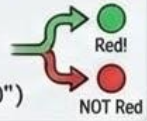
```
detected_color = color_sensor.color(port.B)
if detected_color == 9:  # If red
   await light_matrix.write("RED!")
```

Adding `else` (The Alternative)

- Think: "IF NOT true, THEN do *this* instead"
- Something ALWAYS happens




```
if detected_color == 9:
  await light_matrix.write("RED!")
else: # If NOT red
  await light_matrix.write("NOT RED")
```

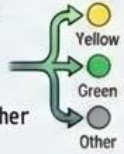


2. Advanced Logic & Reference

Multiple Choices with `elif`

- Think: "else if" - Check another condition
- Handles many possibilities

```
if detected_color == 7:     # Yellow
  motor.run(port.A, 1000)
elif detected_color == 5:   # Green
  motor.run(port.A, -1000)
else:                       # Any other
  motor.stop(port.A)
```



Comparison Operators

Operator	Meaning	Example
==	Equal to	`if color == 7:` `motor.run(port.A, True`
!=	Not equal	`if color != 7:` `motor.run(port.A, True`
>	Greater than	`if color > 5:` `motor.run(port.A, True`
<	Less than	`if color < 7:` `motor.stop(port.A)`

Real Example for Program D

(Putting it together)

```
while True:
  detected_color = color_sensor.color(port.B)
  if detected_color == 7:   # If yellow
    motor.run(port.A, 1000) # Spin motor
  else:                     # If not yellow
    motor.stop(port.A)      # Stop motor

  await runloop.sleep_ms(100)
```

Continuous Loop (`while True:`)

Check Sensor

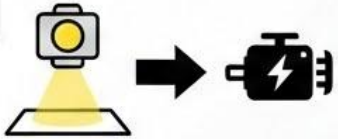
Decision Point

Alternative Action (`else`)

Small Delay

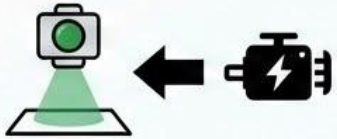
Program E: Advanced Color Sensor Control

✓ Yellow → Motor Forward (1 second)



- When sensor sees yellow (color code 7)
- Motor spins forward at full speed
- Runs for exactly 1 second, then stops

✓ Green → Motor Backward (1 second)



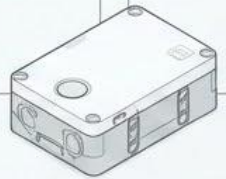
- When sensor sees green (color code 5)
- Motor spins backward at full speed
- Runs for exactly 1 second, then stops

✓ Any Other Color → Display Message

- When sensor sees red, blue, black, white, etc.
- LED matrix displays “CHANGE COLOR”
- Motor does not move

✓ Continuous Checking

- Program runs forever with `while True:` loop
- Constantly monitors color sensor
- Responds immediately to color changes



🏆 SOTC Robotics Team Challenge 🏆

Choose Your Team Level:

🟢 **BEGINNER TEAM**
(New to Python)
→ Earn **1.5×** points

🔵 **ADVANCED TEAM**
(Experienced)
→ Earn **1×** points

Discuss with your partner and choose honestly!

How to Win:

🏆 Complete programs as a team. Highest team score wins!

Bonus Points:

🤝 **Team Helper Bonus:** +5 pts each time your team helps another team (write names on board)

⚙️ **Creativity Bonus:** +10 pts for extending Program E with a creative feature. Examples: Add 3+ colors, make patterns, create a game, add sounds

Score Examples:

🧮 **Beginner Team:** 4 programs + help 2 teams + creativity = 24 + 10 + 10 = **44 pts**

🧮 **Advanced Team:** All 5 programs + help 3 teams + creativity = 20 + 15 + 10 = **45 pts**

Work together. Help others. Be creative. Highest score wins! 🚀

Program	Base Points	🟢 Beginner Team	🔵 Advanced Team
Program A	4 pts	6 pts	4 pts
Program B	4 pts	6 pts	4 pts
Program C	4 pts	6 pts	4 pts
Program D	4 pts	6 pts	4 pts
Program E	4 pts	6 pts	4 pts
TOTAL (All 5)	**20 pts**	**30 pts**	**20 pts**

Session 1: Build & Basic Movement
SPIKE Prime | Grades 6-8

Slide 1: Title

Driving with Signals

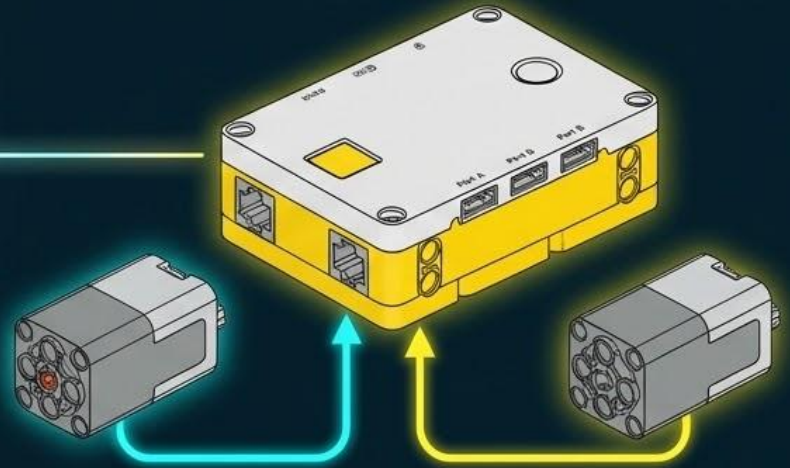
Today: Build a car, make it move, add lights and sound

You'll write **real Python** code!

Slide 2: Build Your Driving Base

Follow the build instructions provided for Driving base 1

Important Setup:



Slide 3

Your Python Driving Base Program

Type this code **exactly:

```
from hub import light_matrix
from hub import motor_pair
import runloop

motor_pair.pair(motor_pair.PAIR_1, 0, 1)

async def main():
    light_matrix.show_image(light_matrix.IMAGE_HAPPY)
    motor_pair.move_for_time(motor_pair.PAIR_1, 2000, 0,
    velocity=50)

runloop.run(main())
```

Run it! What happens?

Understanding the Code

```
from hub import light_matrix
```

Lets us use the lights

```
light_matrix.show_image(...)
```

Shows a picture on the 5x5 display

```
from hub import motor_pair
```

Lets us control both motors together

```
move_for_time(.., 2000, ...)
```

Move for 2000 milliseconds (2 seconds)

```
motor_pair.pair(motor_pair.PAIR_1, 0, 1)
```

Connects Port A (0) and Port B (1)

```
velocity=50
```

Speed (try changing this!)

Confirm this step in the documentation

Try It – Experiment with the Code!

Change **ONE** thing at a time and test:

1. Change velocity

`velocity=50 → velocity=30`

What happens?

2. Change time

`2000 → 4000`

What happens?

3. Change image

`IMAGE_HAPPY → IMAGE_HEART`

What happens?

Write down what you discover!

Understanding Reverse & Sound

What's new?

Sound Library

```
from hub import light_matrix, sound
```

Now we can use sound too!

Reverse Velocity

```
velocity=-50
```

Negative = reverse

Beep Command

```
sound.beep(800, 200, 100)
```

Beep at 800Hz, for 200ms, volume 100

Wait Command

```
await runloop.sleep_ms(500)
```

Wait half a second

Try It - Customize Your Beep

```
sound.beep(frequency, duration, volume)
```

Parameter Explanations

Frequency: pitch (higher number = higher pitch)

Duration: how long in milliseconds

Volume: 0-100

Experiment

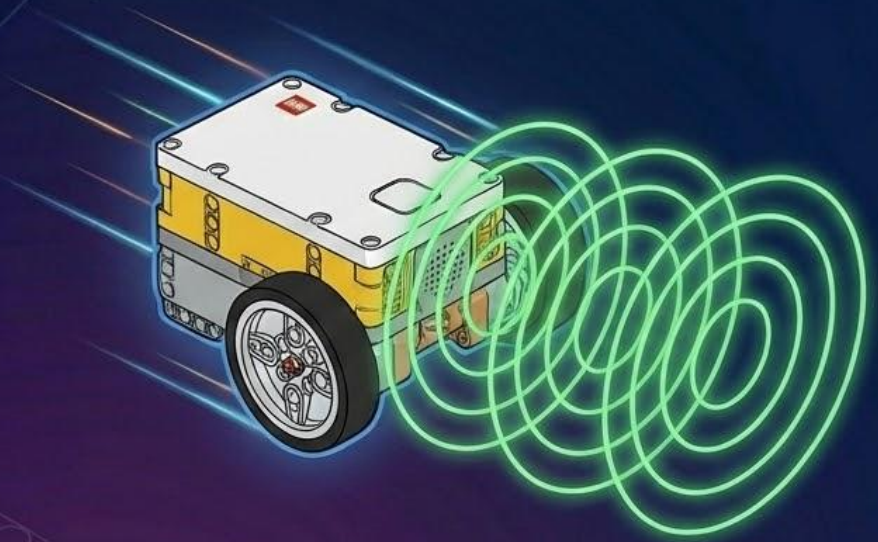
```
sound.beep(400, 500, 100)
```

lower, longer beep

```
sound.beep(1200, 100, 50)
```

higher, shorter, quieter

Repeating Beeps with Loops



```
from hub import light_matrix, sound
from hub import motor_pair
import runloop

motor_pair.pair(motor_pair.PAIR_1, 0, 1)

async def reverse_beep():
    # This loop repeats the beep 4 times
    for i in range(4):
        sound.beep(800, 200, 100)
        await runloop.sleep_ms(400)

async def main():
    # ... (rest of the main function code)
    await reverse_beep()
    # ...
runloop.run(main())
```

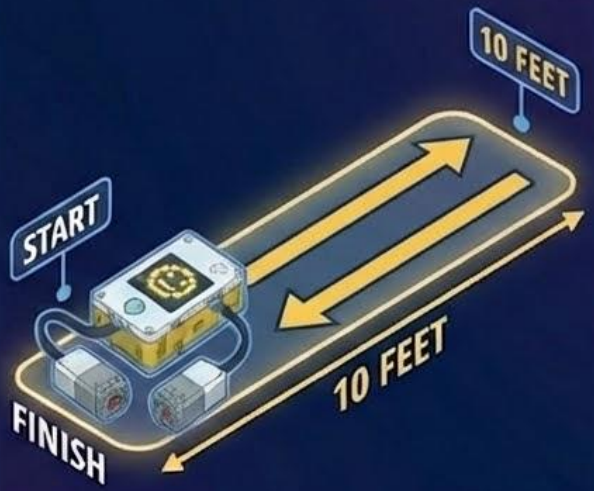
Key Concept: The `for i in range(4):` loop is what makes the beep repeat exactly 4 times.

Final Challenge: The 10-Foot Round Trip

The Mission & Rules

1. Drive 10 feet forward
2. Stop
3. Reverse to start
4. Beep while reversing

Rules: Figure it out, Test & Adjust!



Hints for Success

- Time for 10ft?
- Which velocity?
- Forward time = Reverse time?

Scoring

- ★ 10ft Forward: 5 pts
- ★ Return within 12": 5 pts
- ★ Return within 6": 10 pts
- ★ Beep Reversing: 5 pts

Day 1: The Simulator & Hardware Handshake

Slide 1: THE BUG BOUNTY (0–15 min)

Visual Prompt: **

- **The Mission:** The AI has sabotaged the race files! You must fix the "Red Text" in your app before the garage doors open .
- **Choose Your Side:** * **Program BLUE (The Sprinter):** Drive for time.
    - **Program GOLD (The Spinner):** Turn in a circle.
- **The Rules:** Neighbors **cannot** pick the same side.
- **The Goal:** Fix all syntax errors until the red lines disappear .
- **Support Pod:** Markies, you are the Lead Consultant—ensure the Coach's code is clean!

[Board Snippet: Fix these Bugs!]

`from hub import port light_matrix` (Comma missing!) `await light matrix.write("GO")` (Needs an underscore!) `motor_pair.move_for_time(PAIR_1, 2000, 0, velocity=50)` (Needs `await`!)

Day 1: The Simulator & Hardware Handshake

Slide 1: THE BUG BOUNTY (0–15 min)

➢ **Visual Prompt: ****

- **The Mission:** The AI has sabotaged the race files! You must fix the 'Red Text' in your app before the garage doors open.
- **Choose Your Side:**
  - Program **BLUE** (The Sprinter): Drive for time.
  - Program **GOLD** (The Spinner): Turn in a circle.
- **The Rules:** Neighbors cannot pick the same side.
- **The Goal:** Fix all syntax errors until the red lines disappear.
- **Support Pod:** Markies, you are the Lead Consultant—ensure the Coach's code is clean!

Board Snippet: Fix these Bugs!

```
from hub import port light_matrix ← (Comma missing!)
await light_matrix.write('GO') ← (Needs an underscore!)
motor_pair.move_for_time(PAIR_1, 2000, 0, velocity=50) ← (Needs await!)
```

Slide 2: THE HARDWARE HANDSHAKE (15–35 min)

Visual Prompt: **

- **The Mission:** Match your code to your hardware .
- **Step 1: The Port Check:** Look at your Hub. Are your motors in **A & B** or **C & D**?
- **Step 2: The Update:** Change the letters in your `motor_pair.pair` code to match your physical build .
- **Step 3: The Proof:** Press Play. Your robot must perform its specific Sprint or Spin.
- **The Swap:** Once your robot works, trade seats. Fix your neighbor's ports!

Slide 2: THE HARDWARE HANDSHAKE (15–35 min)

Visual Prompt: **

The Mission: Match your code to your hardware.

Step 1: The Port Check:

Look at your Hub. Are your motors in A & B or C & D?

Step 2: The Update:

`motor_pair.pair(motor_pair.pair` Changethe letters in your code to match your physical build.

Step 3: The Proof:

Press **Play**. Your robot must perform its specific Sprint or Spin.

The Swap: Once your robot works, **trade seats**. Fix your neighbor's ports!

Slide 3: THE DOUBLE-CHECK (35–45 min)

Visual Prompt: **

- **The Requirement:** Both partners must demonstrate their individual code (The Sprint and The Spin) to each other.
  +1
- **Inspector Role:** The Coach and Markies are the official Scrutineers. They are watching the **robot**, not the screen.
- **Pass Criteria:** If the robot moves exactly as coded, you are **"Cleared for the Championship."**

Day 2: The Championship Circuit

Slide 4: CHAMPIONSHIP LOGIC (0–20 min)

Visual Prompt: **

- **The Master Program Sequence:**
  1. **Sprint:** Drive 6 inches (approx. 50% of the target).
  2. **Stop:** Display a "Checkmark" on the light matrix.
     +1
  3. **Sprint:** Drive the remaining distance to hit 12 inches exactly .
  4. **Spin:** 360-degree celebratory turn.
- **Vincent's Mission:** Build and calibrate the **Referee Bot** (Distance Sensor) at the 12-inch mark .
  +1

Slide 5: QUALIFYING TRIALS (20–40 min)

Visual Prompt: **

- **The Test:** Your robot must trigger Vincent’s sensor at exactly 12 inches .
- **The Inspector:** Line up at the Finish Line guarded by **Markies and the Coach**.
- **Stretch Goal:** Program a **Custom Beep** with your own frequency and volume to play during your celebratory spin .
  +1

Slide 6: THE COURT PASS (40–45 min)

Visual Prompt: **

- **The Exit Interview:** Show the Coach your individual app screen.
- **The Question:** Point to the exact line of code that controls your robot's **Velocity** .
- **The Result:** If the Coach approves your code and your 12-inch run, you have earned your **Court Pass!**